Distributions: Boxplots, Violins & Beeswarm

Workshop 3 — Module 2

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

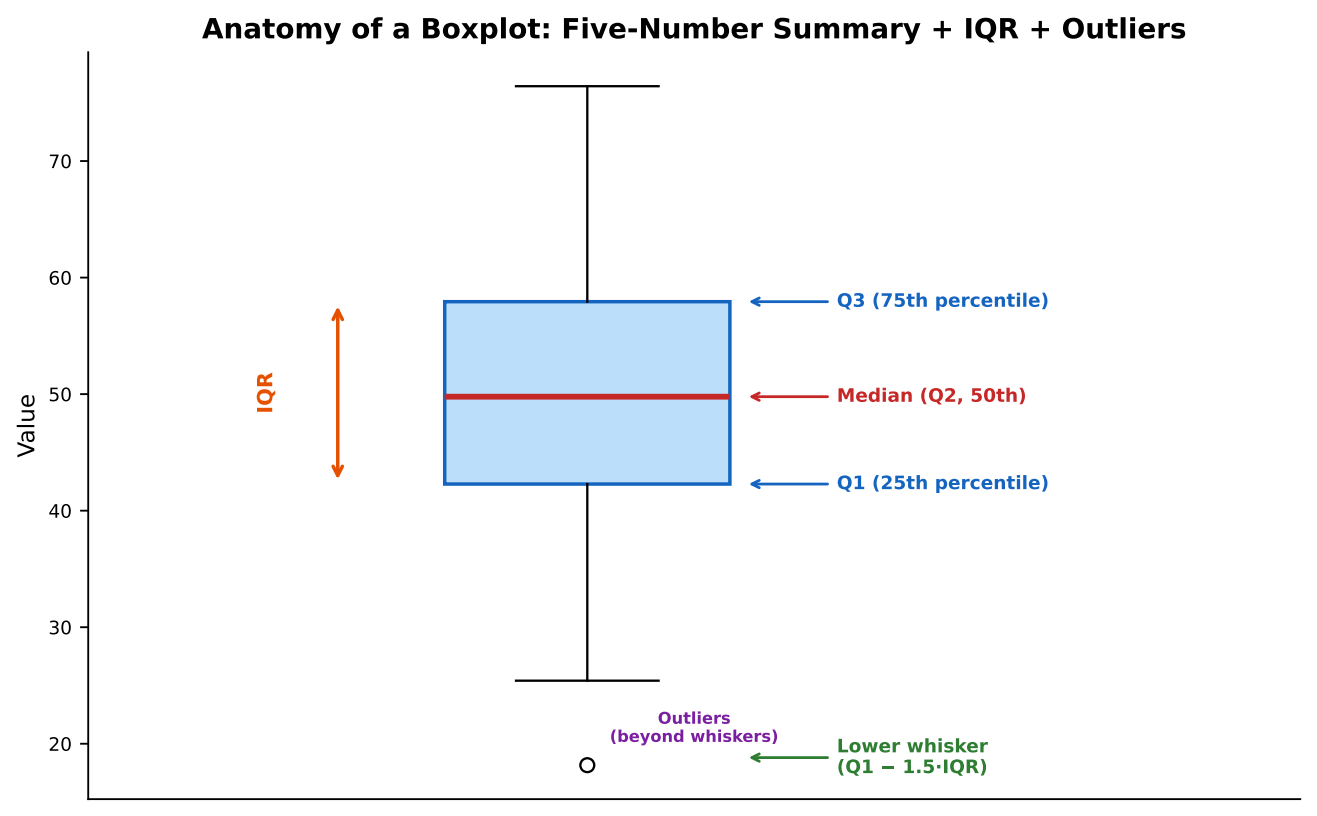
Module Roadmap

1. Boxplot anatomy: the five-number summary
2. What the boxplot hides: shape blindness
3. Violin plots: full density shape
4. Strip, jitter, and beeswarm: showing raw data
5. Raincloud plots: the best of all three
6. Distribution chart selection guide

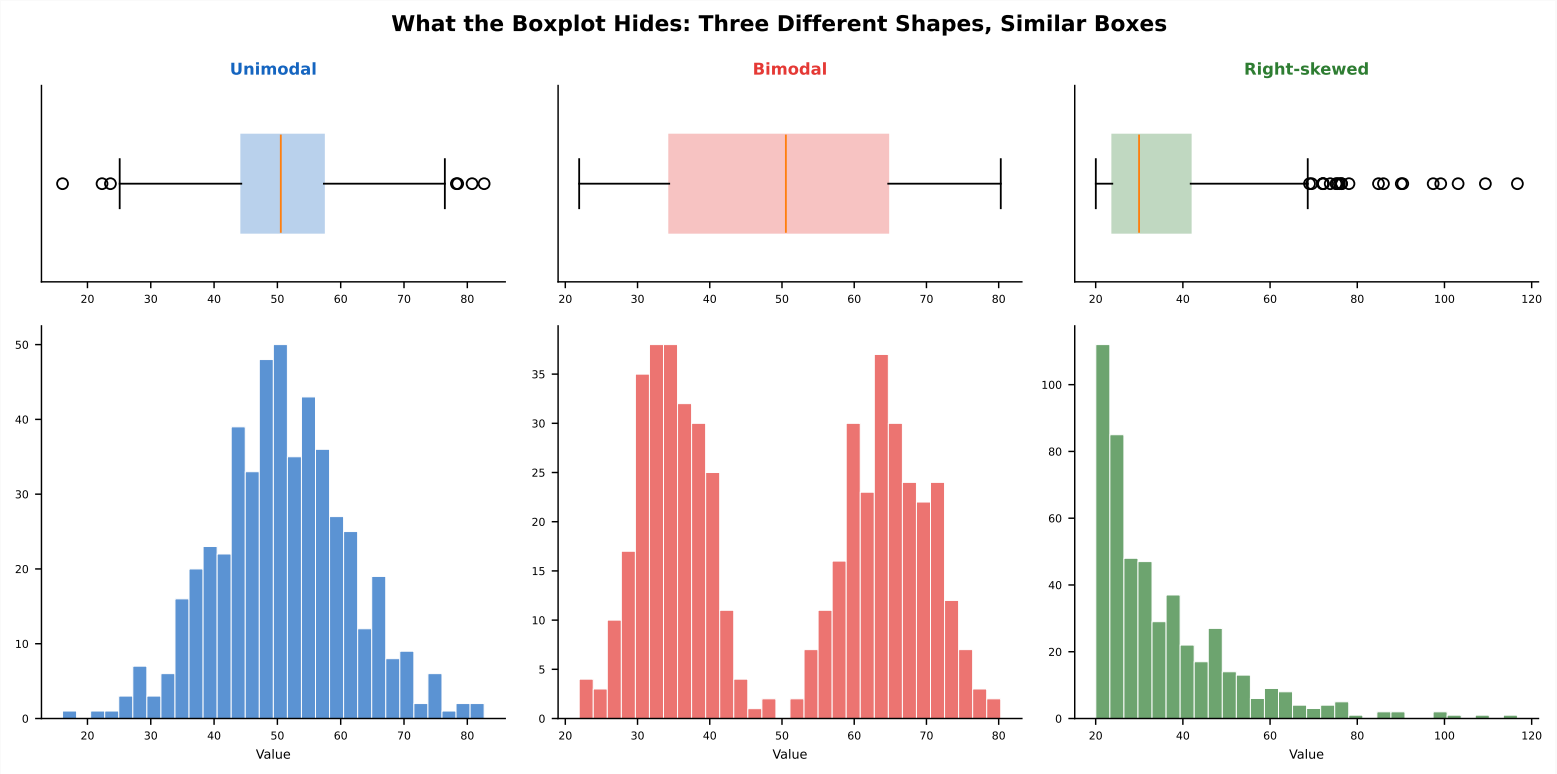
Learning Outcome
You will choose between boxplot, violin, strip, beeswarm, and raincloud based on sample size, audience, and analytical goal.

Boxplot Anatomy

The Five-Number Summary + IQR + Outliers



What the Boxplot Hides



Pro-Tip

Three very different distributions — unimodal, bimodal, and right-skewed — can produce nearly identical boxplots. The box compresses all shape information into five numbers. Always pair a boxplot with jitter, violin, or histogram to show the full picture.

Grouped Boxplot in R vs Python

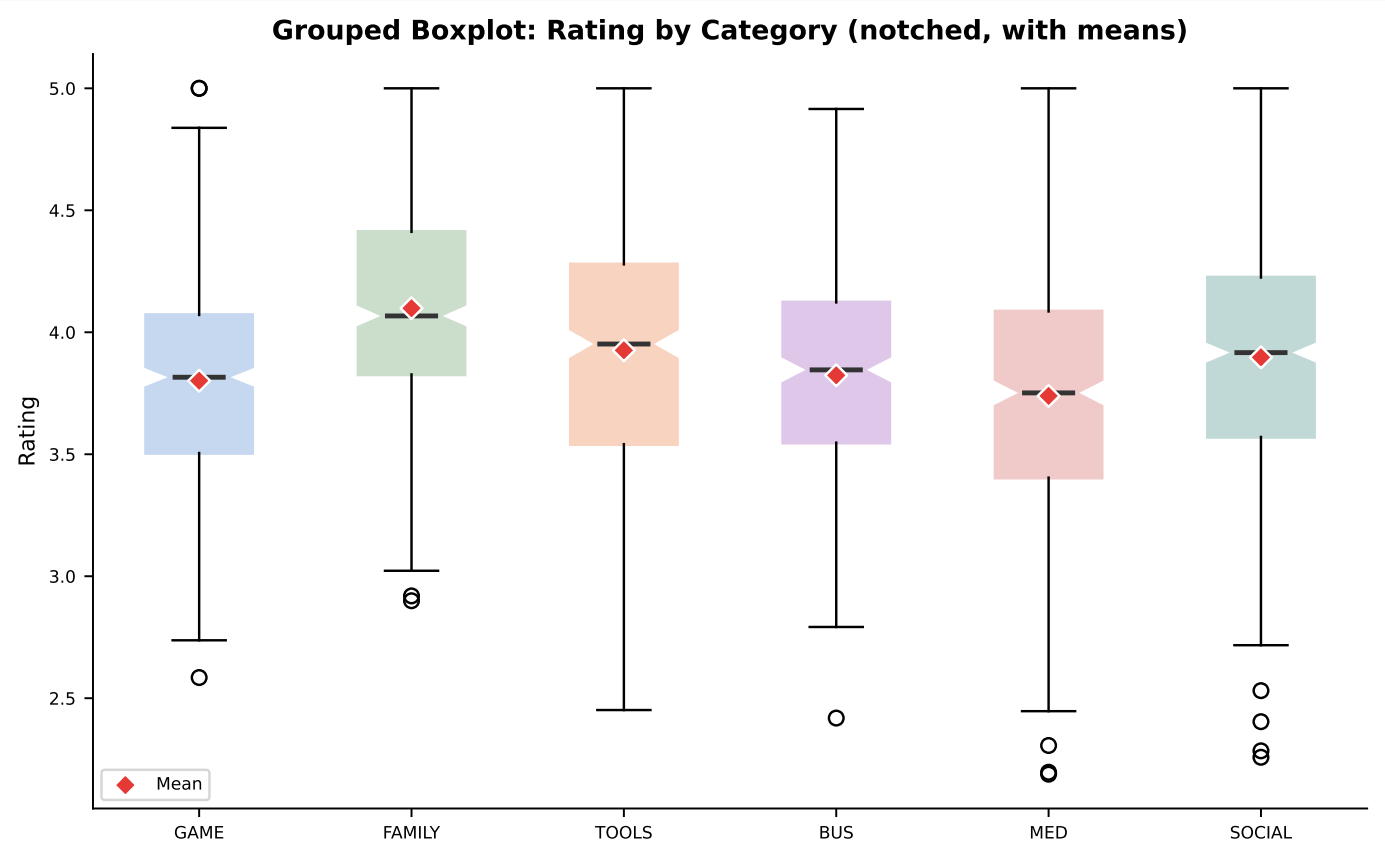
R (ggplot2)

```r
ggplot(apps, aes(x = Category,
    y = Rating, fill = Category))
          +
  geom_boxplot(
    width = 0.5,
    notch = TRUE,
    outlier.alpha = 0.2) +
  stat_summary(fun = mean,
    geom = "point",
    shape = 18, size = 3,
    color = "#E53935") +
  coord_flip() +
  theme_minimal() +
  theme(legend.position = "none")
```

Python (seaborn)

```python
sns.boxplot(data=apps,
    x="Category", y="Rating",
    palette="Set2",
    width=0.5, notch=True,
    flierprops={"alpha": 0.2})

# Or matplotlib:
bp = ax.boxplot(data_list,
    labels=cats,
    patch_artist=True,
    notch=True, widths=0.5)
# Add means:
ax.scatter(range(1,n+1), means,
    c="#E53935", s=40,
    marker="D", zorder=5)
```

Grouped Boxplot with Notch and Means

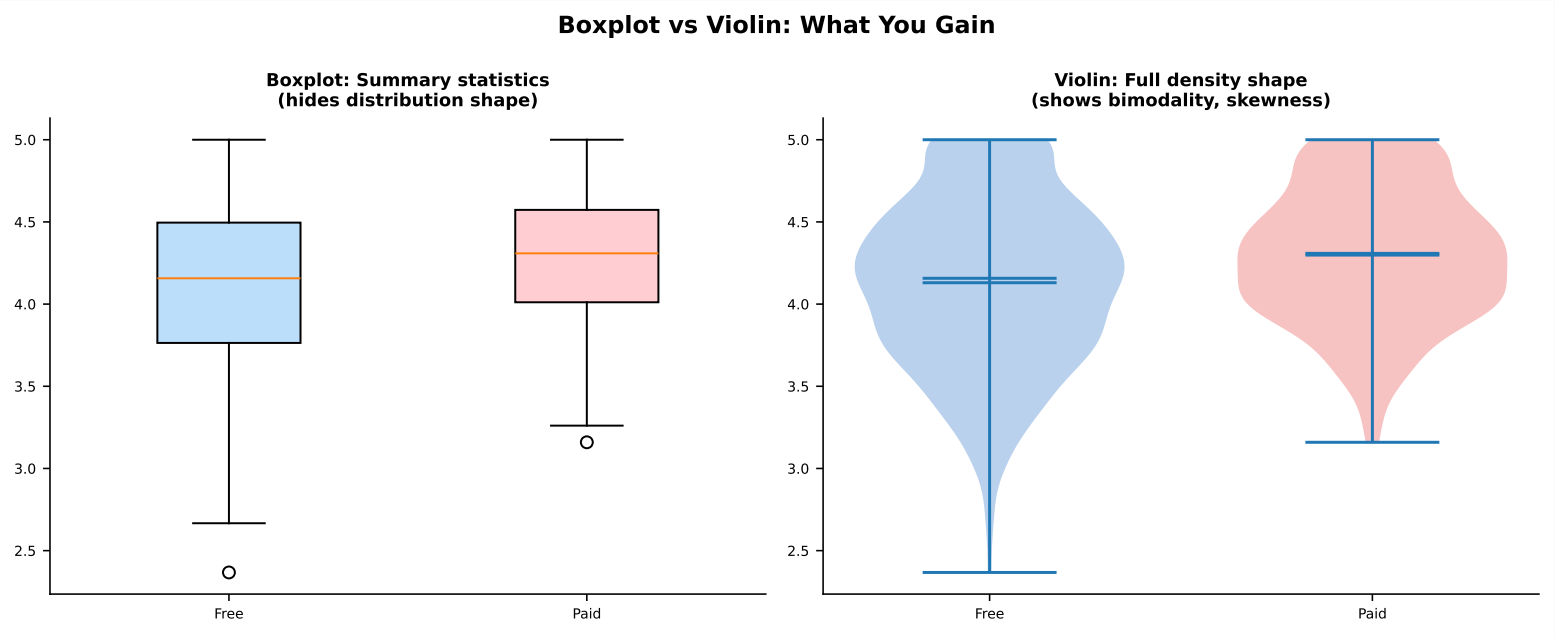


Data Insight

Notches show a confidence interval on the median: if two boxes' notches do not overlap, the medians are significantly different at approximately the 95% level. **Diamond markers** show group means for comparison with medians.

Violin Plots

Boxplot vs Violin: What You Gain



Violin Plots in R vs Python

R

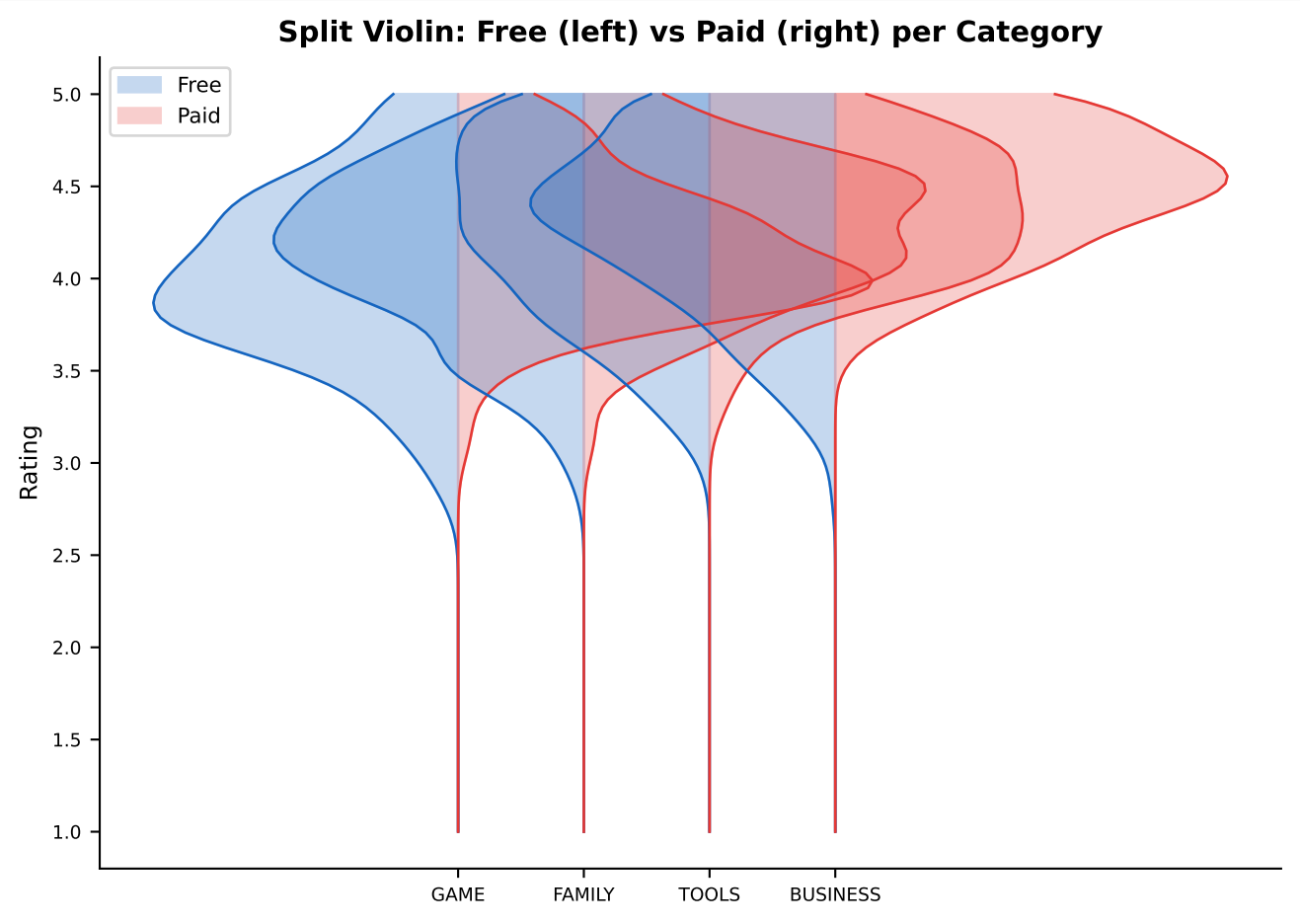
```r
ggplot(apps, aes(x = Type,
    y = Rating, fill = Type)) +
  geom_violin(
    alpha = 0.3,
    draw_quantiles = c(0.25,
      0.5, 0.75)) +
  geom_boxplot(
    width = 0.1,
    outlier.shape = NA,
    fill = "white") +
  scale_fill_manual(values =
    c(Free="#1565C0",
      Paid="#E53935")) +
  theme_minimal() +
  theme(legend.position = "none")
```

Embed a narrow boxplot inside the violin for both shape and summary.

Python

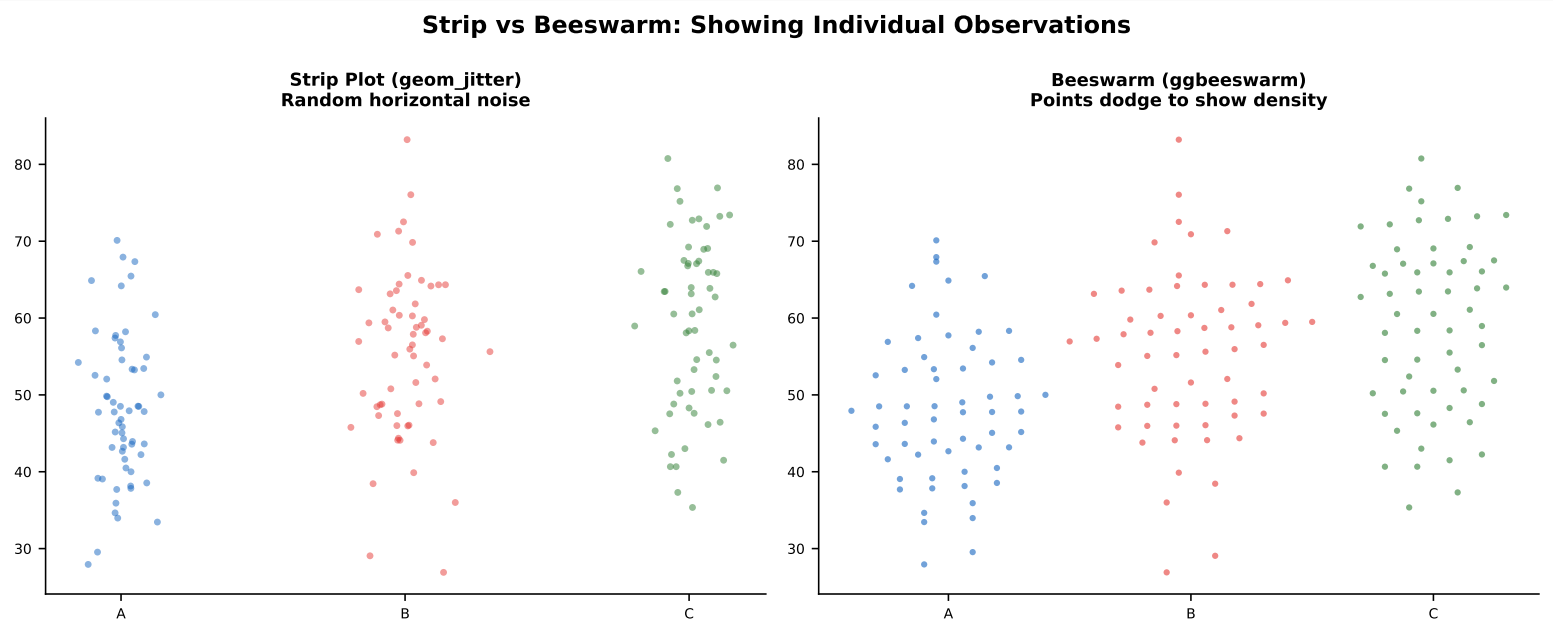
```python
# seaborn (recommended)
sns.violinplot(data=apps,
    x="Type", y="Rating",
    hue="Type",
    palette={"Free":"#1565C0",
             "Paid":"#E53935"},
    inner="quartile",  # or "box"
    split=False, alpha=0.3)

# split=True for side-by-side:
sns.violinplot(data=apps,
    x="Category", y="Rating",
    hue="Type", split=True,
    inner="quart",
    palette={"Free":"#1565C0",
             "Paid":"#E53935"})
```

Split Violin: Two Groups Side by Side



Strip, Jitter & Beeswarm

Showing Individual Observations



Data Insight

Strip/jitter adds random horizontal noise — fast but imprecise. **Beeswarm** stacks points systematically to reveal local density without randomness. Both show every data point, which boxplots and violins hide.

Jitter + Beeswarm in Code

R

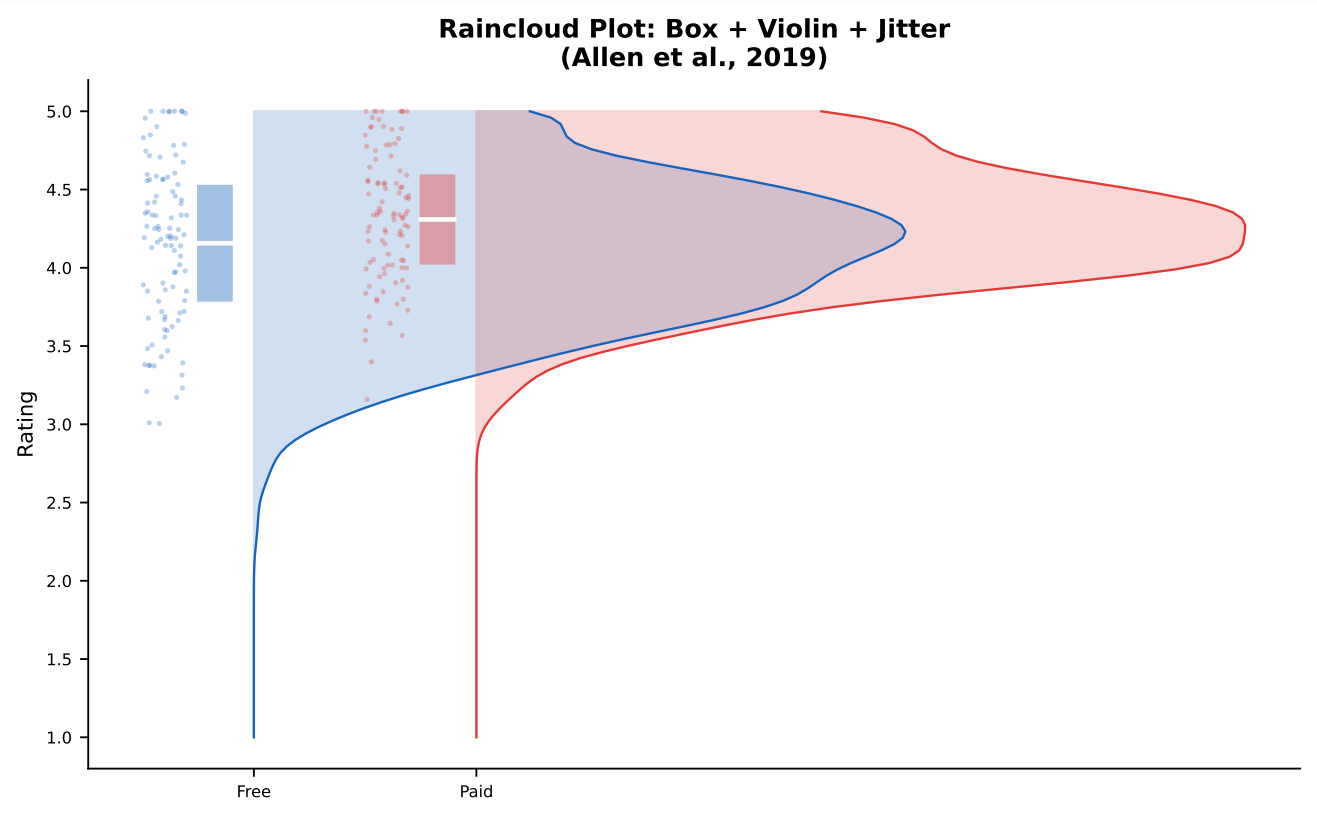
```r
# Jitter
ggplot(apps, aes(x = Type,
    y = Rating)) +
  geom_boxplot(
    outlier.shape = NA,
    fill = "#E3F2FD") +
  geom_jitter(width = 0.15,
    height = 0, alpha = 0.2,
    size = 0.8,
    color = "#E53935") +
  theme_minimal()

# Beeswarm
# install.packages("ggbeeswarm")
library(ggbeeswarm)
ggplot(apps, aes(x = Type,
    y = Rating)) +
  geom_beeswarm(
    cex = 0.5, alpha = 0.3,
    color = "#1565C0", size =
        0.8)
```

Python

```python
# Jitter (strip)
sns.stripplot(data=apps,
    x="Type", y="Rating",
    jitter=0.2, alpha=0.2,
    size=2, color="#E53935")

# Combined: box + strip
fig, ax = plt.subplots()
sns.boxplot(data=apps,
    x="Type", y="Rating",
    width=0.4, ax=ax,
    fliersize=0)
sns.stripplot(data=apps,
    x="Type", y="Rating",
    jitter=0.15, alpha=0.2,
    size=2, color="#E53935",
    ax=ax)

# Beeswarm (seaborn)
sns.swarmplot(data=apps,
    x="Type", y="Rating",
    size=2, alpha=0.3)
```

Raincloud Plots

Raincloud: Box + Violin + Jitter Combined



Pro-Tip

The raincloud plot (Allen et al., 2019) is the gold standard for distribution visualisation in scientific publications. It shows summary statistics (box), density shape (half-violin), and raw data (jitter) simultaneously. In R: `ggrain::geom_rain()`. In Python: manual composition with seaborn.

Chart Selection

Distribution Chart Comparison

Distribution Chart Comparison: Strengths & Weaknesses

Chart	Shows	Weakness	Best For
Boxplot	5-number summary + outliers	Hides shape (bimodality, skew)	Quick comparison across many groups
Violin	Full density shape + mirror symmetry	Hard to read for non-specialists	Show shape when it matters
Strip/Jitter	Individual observations visible	Overplotting with large n	Small n (< 200) per group
Beeswarm	Individual obs + density implied	Slower to render for large n	Moderate n (50-500)
Raincloud	Box + violin + jitter combined	Complex, unfamiliar to some audiences	Maximum information (publications)

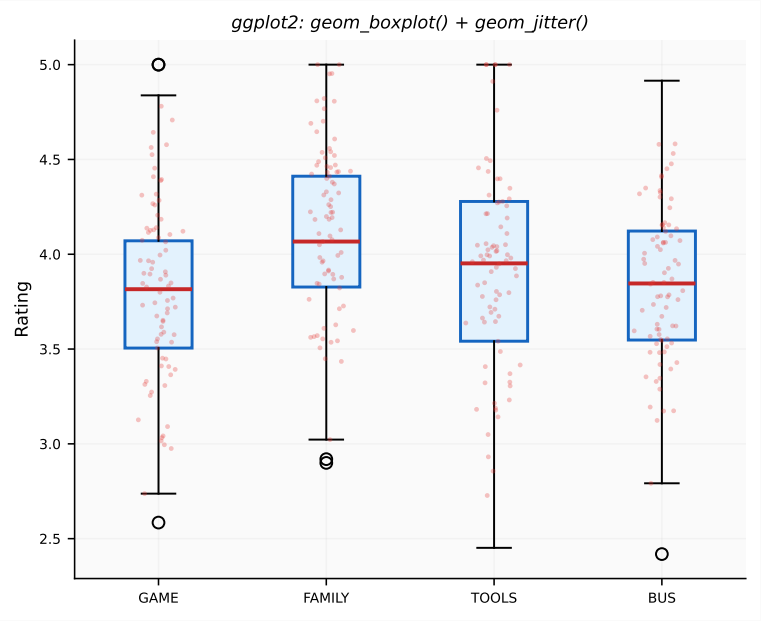
When to Use Which

Scenario	Best Chart	Why
Many groups (>8)	Boxplot	Compact summary
Shape matters	Violin	Shows bimodality
Small n (<200)	Strip / Beeswarm	Every point visible
Publication	Raincloud	Maximum information
Quick EDA	Boxplot + jitter	Fast, informative
Two-group comparison	Split violin	Side-by-side shape

Complete Examples

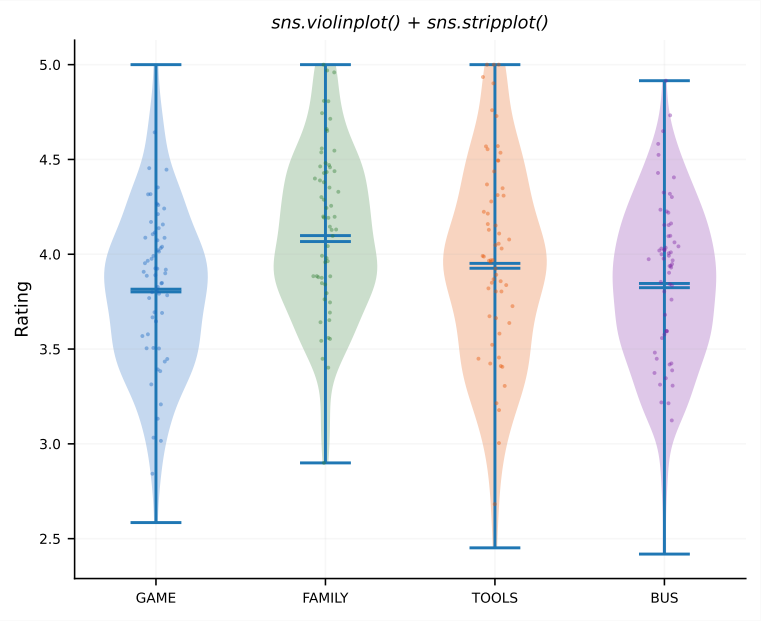
R: Boxplot + Jitter (Publication Ready)

```r
ggplot(apps |>
    filter(Category %in% top4),
  aes(x = fct_reorder(Category,
    Rating, .fun = median),
    y = Rating, fill = Category)) +
  geom_boxplot(
    width = 0.4,
    outlier.shape = NA,
    alpha = 0.3) +
  geom_jitter(
    width = 0.1, alpha = 0.15,
    size = 0.5,
    color = "#E53935") +
  stat_summary(fun = mean,
    geom = "point",
    shape = 18, size = 3) +
  coord_flip() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(title = "Rating by Category"
      ,
       x = NULL, y = "Rating")
```



Python: Violin + Strip

```python
fig, ax = plt.subplots(figsize
    =(6,5))

# Violin layer
parts = ax.violinplot(
    data_list,
    positions=range(1, 5),
    showmeans=True,
    showmedians=True,
    widths=0.6)
for pc, c in zip(
    parts["bodies"], colors):
    pc.set_facecolor(c)
    pc.set_alpha(0.25)

# Strip layer (jitter overlay)
for i, d in enumerate(data_list):
    xj = np.random.normal(
        i + 1, 0.04, min(60, len(d)
            ))
    ax.scatter(xj,
        np.random.choice(d, 60),
        s=4, c=colors[i],
        alpha=0.4, edgecolors="none
            ")
```



Synthesis

Key Takeaways

1. Boxplots show the **five-number summary + outliers** but hide distribution shape (bimodality, skewness).
2. **Violin plots** reveal the full density shape; embed a narrow boxplot inside for both shape and summary.
3. **Strip/jitter** shows every data point; use for small $n < 200$. **Beeswarm** avoids random overlap.
4. **Raincloud** (half-violin + box + jitter) is the publication gold standard (Allen et al., 2019).
5. Always **pair boxplots with jitter or violin** to avoid the shape-blindness problem.
6. In R: `geom_boxplot + geom_jitter`, `geom_violin`, `ggbeeswarm`. In Python: `sns.boxplot + stripplot`, `sns.violinplot`, `sns.swarmplot`.

Essential Readings

- Allen, M. et al. (2019). "Raincloud plots: a multi-platform tool for robust data visualization." *Wellcome Open Research*, 4, 63.
- Wickham, H. & Stryjewski, L. (2011). "40 years of boxplots." *The American Statistician*.
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapters 7, 9 (Distributions).
- Hintze, J. L. & Nelson, R. D. (1998). "Violin plots: a box plot-density trace synergism." *The American Statistician*.

Next Module: Comparisons — Bar Charts, Lollipop & Cleveland Dot (W03-M03)